

# Network Packet Buffer

## 1.Problem Analysis

**Case Story** Router รับ Network Packet เข้ามาเร็วกว่าความสามารถในการส่งออก ดังนั้น Packet จำเป็นต้องถูกเก็บใน Buffer Queue

ในกรณีศึกษาี้ กำหนดให้ Buffer มีขนาดสูงสุด 5 Packets และเปรียบเทียบการจัดการ Buffer ด้วย 2 วิธี คือ

- Algorithm A: Linear Queue
- Algorithm B: Circular Queue

โดยวิเคราะห์ประสิทธิภาพการใช้พื้นที่หน่วยความจำและการจัดการ Packet เมื่อเกิดสถานการณ์ Queue เต็ม

### Input

ข้อมูล Packet ที่เข้าสู่ระบบ ประกอบด้วย

- Packet ID
- Arrival Time
- Size (Bytes)
- Priority

## Output

ผลลัพธ์ที่ระบบต้องแสดง

- สถานะของ Queue ในแต่ละขั้นตอน
- ค่า Front
- ค่า Rear
- Packet ที่ถูกส่งออกจาก Buffer
- Packet ที่ถูกปฏิเสธ (Drop)
- จำนวน Packet ที่อยู่ใน Buffer

## Constraints

ข้อจำกัดของระบบมีดังนี้

1. Buffer สามารถเก็บได้สูงสุด 5 Packets
2. การทำงานต้องเป็นไปตามหลัก FIFO (First In First Out)
3. Packet ที่เข้าก่อนต้องถูกส่งออกก่อน
4. Linear Queue ไม่สามารถนำช่องว่างด้านหน้ากลับมาใช้ใหม่ได้
5. Circular Queue สามารถนำช่องว่างกลับมาใช้ใหม่ได้
6. เมื่อ Buffer เต็มและมี Packet ใหม่เข้ามา ระบบต้องจัดการตามเงื่อนไขของ Algorithm

## Queue

ใช้เก็บข้อมูล Packet ที่กำลังรอส่งออกจากระบบเครือข่าย

แต่ละ Packet มีข้อมูลดังนี้

Packet

└ Packet ID

└ Arrival Time

└ Size

└ Priority

## Suitable Queue Type

### Algorithm A: Linear Queue

โครงสร้างข้อมูลที่ใช้

- Linear Queue

## หลักการ

- เพิ่มข้อมูลทาง Rear
- นำข้อมูลออกทาง Front
- ใช้หลัก FIFO
- เมื่อ Rear ถึงตำแหน่งสุดท้ายของ Array จะไม่สามารถเพิ่มข้อมูลได้อีก

## เหมาะสำหรับ

- ระบบขนาดเล็ก
- จำนวนข้อมูลไม่เปลี่ยนแปลงมาก

## Algorithm B: Circular Queue

### โครงสร้างข้อมูลที่ใช้

- Circular Queue

## หลักการ

- เพิ่มข้อมูลทาง Rear
- นำข้อมูลออกทาง Front
- เมื่อ Rear ถึงตำแหน่งสุดท้าย จะวนกลับไปตำแหน่งแรก

เหมาะสำหรับ

- ระบบ Buffer
- ระบบ Network
- ระบบ Real-Time

เหตุผลที่ต้องใช้ Queue

### 1. รongรับหลัก FIFO

Packet ที่เข้ามาก่อนต้องได้รับการส่งออกก่อน

Arrival:

$P1 \rightarrow P2 \rightarrow P3$

Send:

$P1 \rightarrow P2 \rightarrow P3$

ซึ่งสอดคล้องกับหลักการทำงานของ Queue

## 2. ลดการสูญหายของข้อมูล

เมื่อ Packet เข้ามาเร็วกว่าการส่งออก ระบบสามารถเก็บ Packet ไว้ใน Buffer ก่อน ส่งผลให้ข้อมูลสูญหายน้อยลง

## 3. จำลองการทำงานจริงของ Network

Router, Switch และระบบสื่อสารข้อมูลจริง มักใช้ Queue สำหรับจัดการ Packet ที่รอส่ง

## 4. ใช้เปรียบเทียบประสิทธิภาพของ Linear Queue และ Circular Queue

กรณีศึกษาที่ต้องการแสดงให้เห็นว่า

### Linear Queue

อาจเกิด False Overflow

### Circular Queue

[P6][ ][P3][P4][P5]

↑

Rear

สามารถนำพื้นที่ว่างกลับมาใช้ใหม่ได้ ทำให้ใช้ Buffer ได้คุ้มค่ากว่า

## สรุปการวิเคราะห์ปัญหา

ระบบ Network Packet Buffer ต้องจัดเก็บ Packet ที่รอส่งออกใน Buffer ขนาด 5 ช่อง โดยใช้หลัก FIFO เพื่อรักษาลำดับการส่งข้อมูล การเปรียบเทียบระหว่าง Linear Queue และ Circular Queue จะช่วยแสดงให้เห็นปัญหา False Overflow ของ Linear Queue และประสิทธิภาพการใช้พื้นที่ที่ดีกว่าของ Circular Queue ซึ่งเหมาะสมกับระบบเครือข่ายจริงมากกว่าเนื่องจากสามารถนำพื้นที่ว่างกลับมาใช้ซ้ำได้อย่างมีประสิทธิภาพ

## 2.Queue Design

### Data Structure Selection

ระบบนี้เลือกใช้ Queue เนื่องจาก Network Packet ต้องถูกส่งตามลำดับที่เข้ามา (FIFO: First In First Out)

Packet ที่เข้ามาก่อนจะถูกส่งออกก่อน ทำให้เหมาะกับการใช้ Queue ในการจัดการ Buffer

## Packet Class Design

Packet แต่ละตัวประกอบด้วยข้อมูลดังนี้

Packet

```
-----  
packetId : String  
arrivalTime : int  
size : int  
priority : int  
-----
```

## Algorithm A Design : Linear Queue

Data Structure

```
buffer[5]  
front  
Rear
```

Initial State

```
front = 0  
rear = -1
```

```
buffer = [ ][ ][ ][ ][ ]
```



## หลักการทํางาน

- Enqueue ที่ Rear
- Dequeue ที่ Front
- ใช้ FIFO
- เมื่อ Rear ถึงตำแหน่งสุดท้ายจะไม่สามารถ Enqueue ได้
- อาจเกิด False Overflow

## Algorithm B Design : Circular Queue

### Data Structure

```
buffer[5]  
front  
rear  
count
```

### Initial State

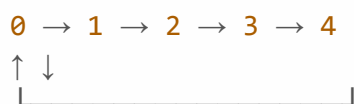
```
front = 0  
rear = -1  
count = 0
```

```
buffer = [ ][ ][ ][ ][ ]
```

## หลักการทำงาน

- Enqueue ที่ Rear
- Dequeue ที่ Front
- ใช้ FIFO
- ตำแหน่ง Rear และ Front สามารถวนกลับมาใช้ใหม่ได้

## Circular Structure



## Design Comparison

| รายการ                       | Linear Queue | Circular Queue |
|------------------------------|--------------|----------------|
| FIFO                         | ✓            | ✓              |
| ใช้พื้นที่ว่างซ้ำ            | ✗            | ✓              |
| False Overflow               | เกิดได้      | ไม่เกิด        |
| เหมาะกับ Buffer              | ปานกลาง      | ดี             |
| ประสิทธิภาพการใช้หน่วยความจำ | ต่ำกว่า      | สูงกว่า        |

### 3.Pseudocode Algorithm A,B

#### Algorithm Design

Algorithm A : Linear Queue Buffer Management

```
Algorithm LINEAR_QUEUE_ENQUEUE(packet)
```

```
if rear = MAX - 1 then
    display "Packet Dropped"
else
    if front = -1 then
        front ← 0
    end if

    rear ← rear + 1
    queue[rear] ← packet
end if
```

End Algorithm

```
Algorithm LINEAR_QUEUE_DEQUEUE()
```

```
if front = -1 OR front > rear then
    display "Queue Empty"
else
    packet ← queue[front]

    front ← front + 1
```

```
    return packet
end if

End Algorithm
```

### Algorithm B : Circular Queue Buffer Management

```
Algorithm CIRCULAR_QUEUE_ENQUEUE(packet)

if count = MAX then
    display "Buffer Full"
else

    rear  $\leftarrow$  (rear + 1) mod MAX

    queue[rear]  $\leftarrow$  packet

    count  $\leftarrow$  count + 1

    if count = 1 then
        front  $\leftarrow$  rear
    end if

end if

End Algorithm
```

```
Algorithm CIRCULAR_QUEUE_DEQUEUE()
```

```
if count = 0 then
```

```
    display "Queue Empty"
```

```
else
```

```
    packet ← queue[front]
```

```
    front ← (front + 1) mod MAX
```

```
    count ← count - 1
```

```
    return packet
```

```
end if
```

```
End Algorithm
```

## Algorithm A : Process Packets

```
Algorithm PROCESS_LINEAR_QUEUE  
  
while queue is not empty  
    packet ← dequeue()  
    send packet  
end while  
  
End Algorithm
```

## Algorithm B : Process Packets

```
Algorithm PROCESS_CIRCULAR_QUEUE  
  
while count > 0  
    packet ← dequeue()  
    send packet  
end while  
  
End Algorithm
```

## Algorithm A (Linear Queue)

- ใช้ FIFO
- เพิ่มข้อมูลที่ Rear
- นำข้อมูลออกที่ Front
- เมื่อ Rear ถึงตำแหน่งสุดท้ายของ Buffer จะไม่สามารถเพิ่ม Packet ใหม่ได้
- อาจเกิด False Overflow

## Algorithm B (Circular Queue)

- ใช้ FIFO เช่นเดียวกัน
- ใช้  $(rear + 1) \bmod MAX$  และ  $(front + 1) \bmod MAX$
- นำช่องว่างที่เกิดจากการ Dequeue กลับมาใช้ใหม่ได้
- ไม่เกิด False Overflow
- เหมาะกับ Network Packet Buffer มากกว่า

## 4.Queue Trace (10 Operations)

### Algorithm A : Linear Queue

**Buffer Size = 5 Packets**

Initial State

Front = -1

Rear = -1

Queue = []

| Step | Operation  | Queue Before | Queue After | Front | Rear | Output |
|------|------------|--------------|-------------|-------|------|--------|
| 1    | ENQUEUE P1 | []           | [P1]        | 0     | 0    | -      |
| 2    | ENQUEUE P2 | [P1]         | [P1,P2]     | 0     | 1    | -      |
| 3    | ENQUEUE P3 | [P1,P2]      | [P1,P2,P3]  | 0     | 2    | -      |
| 4    | DEQUEUE    | [P1,P2,P3]   | [P2,P3]     | 1     | 2    | P1     |
| 5    | DEQUEUE    | [P2,P3]      | [P3]        | 2     | 2    | P2     |
| 6    | ENQUEUE P4 | [P3]         | [P3,P4]     | 2     | 3    | -      |



|    |            |            |            |   |   |         |
|----|------------|------------|------------|---|---|---------|
| 7  | ENQUEUE P5 | [P3,P4]    | [P3,P4,P5] | 2 | 4 | -       |
| 8  | ENQUEUE P6 | [P3,P4,P5] | [P3,P4,P5] | 2 | 4 | Drop P6 |
| 9  | DEQUEUE    | [P3,P4,P5] | [P4,P5]    | 3 | 4 | P3      |
| 10 | ENQUEUE P7 | [P4,P5]    | [P4,P5]    | 3 | 4 | Drop P  |

## อธิบาย

หลัง Step 7 ค่า Rear อยู่ตำแหน่งสุดท้ายของ Array (Index 4)

[ ][ ][P3][P4][P5]  
F R

แม้ว่าด้านหน้าจะมีพื้นที่ว่าง 2 ช่อง แต่ Linear Queue ไม่สามารถนำกลับมาใช้ได้

ดังนั้น

ENQUEUE P6

ENQUEUE P7

จึงถูก Drop ทั้งหมด

นี่คือ **False Overflow**

## Algorithm B : Circular Queue

**Buffer Size = 5 Packets**

Initial State

Front = 0

Rear = -1

Count = 0

Queue = []

| Step | Operation  | Queue Before | Queue After | Front | Rear | Output |
|------|------------|--------------|-------------|-------|------|--------|
| 1    | ENQUEUE P1 | []           | [P1]        | 0     | 0    | -      |
| 2    | ENQUEUE P2 | [P1]         | [P1,P2]     | 0     | 1    | -      |
| 3    | ENQUEUE P3 | [P1,P2]      | [P1,P2,P3]  | 0     | 2    | -      |
| 4    | DEQUEUE    | [P1,P2,P3]   | [P2,P3]     | 1     | 2    | P1     |
| 5    | DEQUEUE    | [P2,P3]      | [P3]        | 2     | 2    | P2     |
| 6    | ENQUEUE P4 | [P3]         | [P3,P4]     | 2     | 3    | -      |

|    |            |                  |                  |   |   |    |
|----|------------|------------------|------------------|---|---|----|
| 7  | ENQUEUE P5 | [P3,P4]          | [P3,P4,P5]       | 2 | 4 | -  |
| 8  | ENQUEUE P6 | [P3,P4,P5]       | [P3,P4,P5,P6]    | 2 | 0 | -  |
| 9  | ENQUEUE P7 | [P3,P4,P5,P6]    | [P3,P4,P5,P6,P7] | 2 | 1 | -  |
| 10 | DEQUEUE    | [P3,P4,P5,P6,P7] | [P4,P5,P6,P7]    | 3 | 1 | P3 |

## สถานะจริงของ Circular Queue หลัง Step 9

Index

0 1 2 3 4

-----

P6 | P7 | P3 | P4 | P5

-----

F

R

หลัง Step 10

Index

0 1 2 3 4

P6 | P7 | | P4 | P5

F

R

## สรุปผล Queue Trace

### Linear Queue

[P3][P4][P5]

- เกิด False Overflow
- ไม่สามารถใช้ช่องว่างด้านหน้าได้
- P6 และ P7 ถูก Drop

### Circular Queue

[P4][P5][P6][P7]

- Rear วนกลับมาใช้ Index 0 และ 1
- ใช้พื้นที่ Buffer ได้ครบทุกช่อง
- ไม่มี False Overflow
- ไม่มี Packet สูญหายจากพื้นที่ว่างที่ยังใช้งานได้

## ข้อสังเกต

จาก Queue Trace จะเห็นว่า **Circular Queue** มีประสิทธิภาพในการใช้ **Buffer** ดีกว่า **Linear Queue** เพราะสามารถนำตำแหน่งที่ว่างจากการ Dequeue กลับมาใช้ซ้ำได้ ทำให้รองรับ Packet ได้มากกว่าและลดโอกาสการ Drop Packet ในระบบ Network Packet Buffer

## 5. Correctness / Invariant

### 5.1 Correctness ของ Algorithm A (Linear Queue)

#### หลักการทำงาน

Algorithm A ใช้โครงสร้างข้อมูล Linear Queue ซึ่งปฏิบัติตามหลัก **FIFO (First In First Out)**

- Packet ใหม่จะถูกเพิ่มที่ตำแหน่ง Rear
- Packet ที่อยู่ตำแหน่ง Front จะถูกนำออกก่อน
- Packet ที่เข้ามาก่อนจะถูกส่งออกก่อนเสมอ

ดังนั้นลำดับการส่ง Packet จะตรงกับลำดับการเข้าของ Packet ทุกครั้ง

## ตัวอย่าง

ENQUEUE P1

ENQUEUE P2

ENQUEUE P3

Queue = [P1, P2, P3]

## เมื่อดำเนินการ

DEQUEUE

## ผลลัพธ์

Output = P1

Queue = [P2, P3]

แสดงว่า Packet ตัวแรกที่เข้าระบบถูกนำออกก่อน จึงเป็นไปตามหลัก FIFO

## Correctness Statement

สำหรับทุก Packet ที่อยู่ใน Queue หาก Packet A เข้าสู่ Queue ก่อน Packet B และทั้งสอง Packet ยังไม่ถูกลบออกจากระบบ Algorithm A จะนำ Packet A ออกจาก Queue ก่อน Packet B เสมอ

ดังนั้น Algorithm A ทำงานถูกต้องตามหลัก FIFO ของ Queue

## 5.2 Invariant ของ Algorithm A

### Invariant

ก่อนเริ่มแต่ละรอบของ Algorithm สมาชิกที่ตำแหน่ง Front คือ Packet ที่เข้ามาก่อนที่สุดในบรรดา Packet ทั้งหมดที่ยังไม่ได้รับการส่งออก

### พิสูจน์ Invariant

#### Initialization

ก่อนมีการทำงานใด ๆ

Queue = Empty

Invariant เป็นจริง

#### Maintenance

เมื่อ ENQUEUE

$\text{Rear} \leftarrow \text{Rear} + 1$

Packet ใหม่ถูกเพิ่มท้าย Queue Packet ที่ Front ยังคงเป็น Packet ที่เข้าก่อนที่สุด Invariant ยังคงเป็นจริง เมื่อ DEQUEUE

Remove Packet at Front

Packet ลำดับถัดไปจะเลื่อนขึ้นมาเป็น Front และเป็น Packet ที่เข้าก่อนที่สุดใน Queue ที่เหลือ Invariant ยังคงเป็นจริง

## Termination

เมื่อ Queue ว่าง

Front > Rear

Packet ทุกตัวได้รับการประมวลผลครบถ้วนตามลำดับ FIFO

ดังนั้น Algorithm A ถูกต้อง



## 5.3 Correctness ของ Algorithm B (Circular Queue)

### หลักการทำงาน

Circular Queue ยังคงใช้หลัก FIFO เหมือน Linear Queue

ความแตกต่างคือ

```
rear = (rear + 1) mod MAX  
front = (front + 1) mod MAX
```

เพื่อให้ตำแหน่งสามารถวนกลับมาใช้ใหม่ได้

แม้ตำแหน่ง Front และ Rear จะเปลี่ยนแบบวงกลม แต่ลำดับการเข้าของ Packet ยังคงเดิม

### ตัวอย่าง

```
ENQUEUE P1  
ENQUEUE P2  
ENQUEUE P3
```

### Queue

```
[P1, P2, P3]
```

เมื่อ

```
DEQUEUE
```

ผลลัพธ์

Output = P1

หลังจากนั้น

ENQUEUE P4

ENQUEUE P5

ตำแหน่ง Rear อาจวนกลับมาใช้ช่องเดิม แต่ลำดับการส่งข้อมูลยังเป็น

$P2 \rightarrow P3 \rightarrow P4 \rightarrow P5$

ซึ่งยังเป็น FIFO

### Correctness Statement

สำหรับทุก Packet ที่อยู่ใน Circular Queue หาก Packet A เข้าสู่ระบบก่อน Packet B และทั้งสอง Packet ยังไม่ถูกส่งออก Packet A จะถูกส่งออกก่อน Packet B เสมอ

ดังนั้น Circular Queue ยังคงรักษาหลัก FIFO ได้อย่างถูกต้อง

## 5.4 Invariant ของ Algorithm B

### Invariant

ก่อนเริ่มแต่ละรอบของ Algorithm ตำแหน่ง Front จะชี้ไปยัง Packet ที่เข้ามาก่อนที่สุดในบรรดา Packet ทั้งหมดที่ยังอยู่ใน Buffer และ Count จะเท่ากับจำนวน Packet ที่อยู่ใน Queue จริงเสมอ

### พิสูจน์ Invariant

#### Initialization

เริ่มต้น

```
front = 0  
rear = -1  
count = 0
```

Queue ว่าง

Invariant เป็นจริง

## Maintenance

เมื่อ ENQUEUE

```
rear = (rear + 1) mod MAX  
count++
```

Packet ถูกเพิ่มท้าย Queue Packet ที่ Front ไม่เปลี่ยน Invariant ยังคงเป็นจริง

เมื่อ DEQUEUE

```
front = (front + 1) mod MAX  
count--
```

Packet ตัวแรกถูกนำออก Packet ที่เข้าก่อนที่สุดลำดับถัดไปจะกลายเป็น Front ใหม่ Invariant ยังคงเป็นจริง

## Termination

เมื่อ

```
count = 0
```

หมายถึงไม่มี Packet เหลือใน Buffer Packet ทุกตัวถูกประมวลผลครบตามลำดับ FIFO ดังนั้น Algorithm B ทำงานถูกต้อง

## สรุป Correctness

- **Algorithm A (Linear Queue)** ทำงานถูกต้องเพราะนำ Packet ออกตามลำดับที่เข้ามา (FIFO)
- **Algorithm B (Circular Queue)** ทำงานถูกต้องเช่นเดียวกัน และสามารถใช้พื้นที่ Buffer ซ้ำได้โดยไม่กระทบลำดับของ Packet
- Invariant ของทั้งสองอัลกอริทึมรับประกันว่า Packet ที่ตำแหน่ง Front จะเป็น Packet ที่เข้ามาก่อนที่สุดในบรรดา Packet ที่ยังไม่ได้ถูกส่งออก ทำให้การทำงานถูกต้องตามข้อกำหนดของระบบ Network Packet Buffer เสมอ

## 6. Time Complexity

Time Complexity คือการวิเคราะห์จำนวนการทำงานของ Algorithm เมื่อขนาดข้อมูล ( $n$ ) เพิ่มขึ้น

โดยในระบบ Network Packet Buffer จะพิจารณา Operation หลักของ Queue ได้แก่

- enqueue()
- dequeue()
- peek()
- search()
- display()

## 6.1 Algorithm A : Linear Queue

### Enqueue

เพิ่ม Packet ที่ตำแหน่ง Rear

```
rear ← rear + 1  
queue[rear] ← packet
```

ไม่ต้องวนลูปค้นหาตำแหน่ง

### Time Complexity

$O(1)$

### Dequeue

นำ Packet ที่ตำแหน่ง Front ออก

```
packet ← queue[front]  
front ← front + 1
```

เข้าถึงข้อมูลโดยตรง

### Time Complexity

$O(1)$

## Peek

ดู Packet ตัวแรกโดยไม่ลบออก

```
queue[front]
```

เข้าถึงได้ทันที

## Time Complexity

$O(1)$

## Search

ค้นหา Packet ตาม Packet ID

ตัวอย่างค้นหา P5

```
[P1][P2][P3][P4][P5]
```

อาจต้องตรวจสอบทุก Packet กรณีเลขร้ายที่สุดค้นหาตัวสุดท้าย

## Time Complexity

$O(n)$

## Display

แสดงข้อมูล Packet ทุกตัวใน Queue

```
[P1][P2][P3][P4][P5]
```

ต้องแสดงครบทุกสมาชิก

## Time Complexity

$O(n)$

### สรุป Time Complexity ของ Linear Queue

| Operation | Complexity |
|-----------|------------|
| enqueue() | $O(1)$     |
| dequeue() | $O(1)$     |
| peek()    | $O(1)$     |
| search()  | $O(n)$     |
| display() | $O(n)$     |



## 6.2 Algorithm B : Circular Queue

แม้ว่าจะมีการคำนวณ

```
(rear + 1) mod MAX  
(front + 1) mod MAX
```

แต่เป็นการคำนวณค่าคงที่

จึงไม่ส่งผลต่อระดับ Complexity

### Enqueue

```
rear ← (rear + 1) mod MAX
```

### Time Complexity

$O(1)$

### Dequeue

```
front ← (front + 1) mod MAX
```

### Time Complexity

$O(1)$

### Peek

เข้าถึงสมาชิกตัวแรก

```
queue[front]
```

## Time Complexity

$O(1)$

## Search

ตรวจสอบ Packet ที่ละตัว

## Time Complexity

$O(n)$

## Display

แสดงสมาชิกทั้งหมดใน Queue

## Time Complexity

$O(n)$

## สรุป Time Complexity ของ Circular Queue

| Operation | Complexity |
|-----------|------------|
| enqueue() | $O(1)$     |
| dequeue() | $O(1)$     |
| peek()    | $O(1)$     |
| search()  | $O(n)$     |
| display() | $O(n)$     |

## สรุปเปรียบเทียบ Time Complexity

| Operation | Linear Queue | Circular Queue |
|-----------|--------------|----------------|
| enqueue() | $O(1)$       | $O(1)$         |
| dequeue() | $O(1)$       | $O(1)$         |
| peek()    | $O(1)$       | $O(1)$         |
| search()  | $O(n)$       | $O(n)$         |
| display() | $O(n)$       | $O(n)$         |

ถึงแม้ Time Complexity จะเท่ากัน แต่ Circular Queue ใช้พื้นที่ Buffer ได้มีประสิทธิภาพกว่า เพราะสามารถนำตำแหน่งที่ว่างกลับมาใช้งานได้และไม่เกิด False Overflow

## 7.Space Complexity

Space Complexity คือการวิเคราะห์ปริมาณหน่วยความจำที่ Algorithm ใช้ระหว่างการทำงาน

กำหนดให้

$n$  = จำนวน Packet ภายใน Queue

### 7.1 Algorithm A : Linear Queue

ใช้พื้นที่สำหรับเก็บ Packet ใน Buffer

`buffer[n]`

และตัวแปรเพิ่มเติม

`front`

`rear`

ซึ่งเป็นค่าคงที่

ดังนั้นพื้นที่ส่วนใหญ่ขึ้นอยู่กับจำนวน Packet ที่เก็บ

**Space Complexity**

$O(n)$

## Auxiliary Space

ตัวแปรเพิ่มเติม

front

rear

ใช้พื้นที่คงที่

$O(1)$

Total Space

$O(n)$

## 7.2 Algorithm B : Circular Queue

ใช้พื้นที่เก็บ Packet

buffer[n]

และตัวแปร

front

rear

count

เป็นค่าคงที่ จึงได้

**Space Complexity**

$O(n)$

## Auxiliary Space

front

rear

count

มีจำนวนคงที่

$O(1)$

## Total Space

$O(n)$

## สรุป Space Complexity

| รายการ          | Linear Queue | Circular Queue |
|-----------------|--------------|----------------|
| Packet Storage  | $O(n)$       | $O(n)$         |
| Auxiliary Space | $O(1)$       | $O(1)$         |
| Total Space     | $O(n)$       | $O(n)$         |

## บทสรุป

Algorithm A (Linear Queue) และ Algorithm B (Circular Queue) มี Time Complexity และ Space Complexity เท่ากัน คือ

Enqueue =  $O(1)$

Dequeue =  $O(1)$

Peek =  $O(1)$

Search =  $O(n)$

Display =  $O(n)$

Space Complexity =  $O(n)$

## 8.Java Implementation

### 8.1 การออกแบบ Class

#### Class Packet

ใช้เก็บข้อมูลของ Packet แต่ละตัวในระบบ Network Packet Buffer

```
class Packet {
    private String packetId;
    private int arrivalTime;
    private int size;
    private int priority;

    public Packet(String packetId, int arrivalTime, int
size, int priority) {
        this.packetId = packetId;
        this.arrivalTime = arrivalTime;
        this.size = size;
        this.priority = priority;
    }

    public String getPacketId() {
        return packetId;
    }

    public int getArrivalTime() {
        return arrivalTime;
    }
}
```



```
public int getSize() {  
    return size;  
}  
  
public int getPriority() {  
    return priority;  
}  
  
@Override  
public String toString() {  
    return packetId;  
}  
}
```

## 8.2 Algorithm A : Linear Queue Implementation

```
class LinearQueueBuffer {  
  
    private Packet[] buffer;  
    private int front;  
    private int rear;  
    private int capacity;  
  
    public LinearQueueBuffer(int capacity) {  
        this.capacity = capacity;  
        buffer = new Packet[capacity];  
        front = 0;  
        rear = -1;  
    }  
  
    public boolean enqueue(Packet packet) {  
  
        if (rear == capacity - 1) {  
            System.out.println(packet + " Dropped (Buffer Full)");  
            return false;  
        }  
  
        buffer[++rear] = packet;  
        return true;  
    }  
  
    public Packet dequeue() {
```

```
if (front > rear) {
    System.out.println("Queue Empty");
    return null;
}

return buffer[front++];
}

public void display() {

    if (front > rear) {
        System.out.println("Queue Empty");
        return;
    }

    for (int i = front; i <= rear; i++) {
        System.out.print(buffer[i] + " ");
    }
    System.out.println();
}

public int getFront() {
    return front;
}

public int getRear() {
    return rear;
}
}
```

## 8.3 Algorithm B : Circular Queue Implementation

```
class CircularQueueBuffer {  
  
    private Packet[] buffer;  
    private int front;  
    private int rear;  
    private int count;  
    private int capacity;  
  
    public CircularQueueBuffer(int capacity) {  
  
        this.capacity = capacity;  
  
        buffer = new Packet[capacity];  
  
        front = 0;  
        rear = -1;  
        count = 0;  
    }  
  
    public boolean enqueue(Packet packet) {  
  
        if (count == capacity) {  
            System.out.println(packet + " Dropped (Buffer Full)");  
            return false;  
        }  
    }  
}
```

```
rear = (rear + 1) % capacity;

buffer[rear] = packet;

count++;

return true;
}

public Packet dequeue() {

    if (count == 0) {
        System.out.println("Queue Empty");
        return null;
    }

    Packet temp = buffer[front];

    front = (front + 1) % capacity;

    count--;

    return temp;
}

public void display() {

    if (count == 0) {
        System.out.println("Queue Empty");
```

```
return;
}

int index = front;

for (int i = 0; i < count; i++) {

    System.out.print(buffer[index] + " ");

    index = (index + 1) % capacity;
}

System.out.println();
}

public int getFront() {
    return front;
}

public int getRear() {
    return rear;
}
}
```

## 8.4 Main Program

ใช้จำลอง Scenario ที่อาจารย์กำหนด

```
public class NetworkPacketBuffer {  
  
    public static void main(String[] args) {  
  
        CircularQueueBuffer queue =  
            new CircularQueueBuffer(5);  
  
        queue.enqueue(new Packet("P1", 1, 500, 1));  
        queue.enqueue(new Packet("P2", 2, 300, 2));  
        queue.enqueue(new Packet("P3", 3, 200, 1));  
  
        queue.dequeue();  
        queue.dequeue();  
  
        queue.enqueue(new Packet("P4", 4, 600, 3));  
        queue.enqueue(new Packet("P5", 5, 400, 2));  
        queue.enqueue(new Packet("P6", 6, 700, 1));  
  
        queue.display();  
  
        System.out.println("Front = " + queue.getFront());  
        System.out.println("Rear = " + queue.getRear());  
    }  
}
```

ผลลัพธ์

```
P3 P4 P5 P6  
Front = 2  
Rear = 0
```

## 8.5 เหตุผลในการเลือกโครงสร้างข้อมูล

### Algorithm A

เลือกใช้

Packet[]

เพื่อจำลองการทำงานของ Linear Queue แบบ Array เนื่องจาก  
สามารถแสดงปัญหา False Overflow ได้อย่างชัดเจน

### Algorithm B

เลือกใช้

Packet[]



ร่วมกับ

front

rear

Count

เพื่อจำลอง Circular Queue และแสดงการนำพื้นที่ว่างกลับมาใช้งานใหม่โดยใช้การคำนวณแบบ Modulo

$(\text{rear} + 1) \% \text{capacity}$

$(\text{front} + 1) \% \text{capacity}$

สรุป

โปรแกรม Java ถูกออกแบบเป็น 3 ส่วนหลัก คือ

1. **Packet** สำหรับเก็บข้อมูล Packet
2. **LinearQueueBuffer** สำหรับ Algorithm A
3. **CircularQueueBuffer** สำหรับ Algorithm B

การออกแบบดังกล่าวสอดคล้องกับ Case Study Network Packet Buffer และสามารถเปรียบเทียบประสิทธิภาพระหว่าง Linear Queue และ Circular Queue ได้อย่างชัดเจน โดยเฉพาะประเด็น False Overflow และการใช้พื้นที่ Buffer อย่างมีประสิทธิภาพ

## 9.Test Cases

### Test Case 1 : Normal Case

#### วัตถุประสงค์

ทดสอบการทำงานปกติของ Queue ในกรณี Enqueue และ Dequeue

#### Input

```
ENQUEUE P1  
ENQUEUE P2  
ENQUEUE P3  
DEQUEUE
```

#### Expected Output

Removed Packet : P1

Queue :  
[P2, P3]

#### ผลที่คาดหวัง

- Packet P1 ถูกนำออกก่อน
- Queue เหลือ P2 และ P3
- ทำงานตามหลัก FIFO

## Test Case 2 : Empty Queue

### วัตถุประสงค์

ทดสอบการ Dequeue จาก Queue ว่าง

### Input

DEQUEUE

### Expected Output

Queue Empty

### ผลที่คาดหวัง

- ระบบไม่ล่ม (No Crash)
- แสดงข้อความ Queue Empty
- ไม่มีข้อมูลถูกลบ

## Test Case 3 : Single Item

### วัตถุประสงค์

ทดสอบ Queue ที่มี Packet เพียง 1 ตัว

### Input

ENQUEUE P1

DEQUEUE

## Expected Output

Removed Packet : P1

Queue :

[]

## ผลที่คาดหวัง

- สามารถเพิ่มและลบข้อมูลได้ถูกต้อง
- Queue กลับเป็นว่าง

## Test Case 4 : Large Queue

### วัตถุประสงค์

ทดสอบประสิทธิภาพเมื่อมีข้อมูลจำนวนมาก

### Input

ENQUEUE 50,000 Packets

P1 ... P50000

### Expected Output

Total Packets = 50000

Processing Completed

## ผลที่คาดหวัง

- โปรแกรมทำงานได้ครบทุก Packet
- ไม่มีข้อมูลสูญหาย
- ใช้สำหรับเก็บค่า Execution Time ในการทดลอง

## Test Case 5 : Special / Edge Case

### วัตถุประสงค์

ทดสอบกรณี Buffer เต็ม

### Input

Buffer Size = 5

ENQUEUE P1

ENQUEUE P2

ENQUEUE P3

ENQUEUE P4

ENQUEUE P5

ENQUEUE P6

### Expected Output (Linear Queue)

P6 Dropped (Buffer Full)

### Expected Output (Circular Queue)

Buffer Full

หรือ

P6 Dropped

ถ้ายังไม่มีช่องว่างให้ใช้งาน

**ผลที่คาดหวัง**

- ระบบตรวจจับสถานะ Queue เต็มได้ถูกต้อง
- ไม่มี Array Index Out Of Bounds Error

**Test Case 6 : Cancel Case**

**วัตถุประสงค์**

ทดสอบการยกเลิก Packet ที่อยู่ระหว่างรอใน Queue

**Input**

ENQUEUE P1

ENQUEUE P2

ENQUEUE P3

CANCEL P2

**Expected Output**

Packet P2 Cancelled

Queue :

[P1, P3]

## ผลที่คาดหวัง

- ระบบสามารถค้นหา Packet ที่ต้องการยกเลิกได้
- Packet ถูกลบออกจาก Queue
- Packet อื่นยังคงลำดับเดิม

## สรุป Test Cases

| Test Case | รายละเอียด        | ผลที่ต้องการ             |
|-----------|-------------------|--------------------------|
| TC1       | Normal Case       | FIFO ทำงานถูกต้อง        |
| TC2       | Empty Queue       | แสดง Queue Empty         |
| TC3       | Single Item       | เพิ่มและลบได้ถูกต้อง     |
| TC4       | Large Queue       | รองรับข้อมูลจำนวนมาก     |
| TC5       | Special/Edge Case | ตรวจจับ Queue เต็ม       |
| TC6       | Cancel Case       | ยกเลิก Packet ได้ถูกต้อง |

## บทสรุป

ชุดทดสอบทั้ง 6 กรณีครอบคลุมการทำงานหลักของระบบ Network Packet Buffer ได้แก่ การทำงานปกติ การจัดการ Queue ว่าง กรณีมีข้อมูลเพียงรายการเดียว การทดสอบข้อมูลจำนวนมาก กรณี Buffer เต็ม และการยกเลิก Packet ที่รออยู่ใน Queue ซึ่งช่วยยืนยันความถูกต้องและความเสถียรของทั้ง Algorithm A (Linear Queue) และ Algorithm B (Circular Queue) ได้อย่างครบถ้วนตามข้อกำหนดของงาน

## 10.Experimental Comparison

### 10.1 วัตถุประสงค์การทดลอง

เพื่อเปรียบเทียบประสิทธิภาพของ

- Algorithm A : Linear Queue
- Algorithm B : Circular Queue

ในระบบ Network Packet Buffer โดยวัดเวลาในการประมวลผล (Execution Time) ของการ Enqueue และ Dequeue Packet จำนวนมาก

### 10.2 วิธีการทดลอง

สภาพแวดล้อมการทดลอง

- ภาษา Java
- ใช้ `System.nanoTime()`
- Warm-up ก่อนการวัดผล
- ใช้ Random Seed คงที่
- ทดสอบ 5 รอบ
- ใช้ค่าเฉลี่ย (Average Time)

ขนาดข้อมูลที่ใช้

$n = 100$

$n = 1,000$

$n = 10,000$

$n = 50,000$



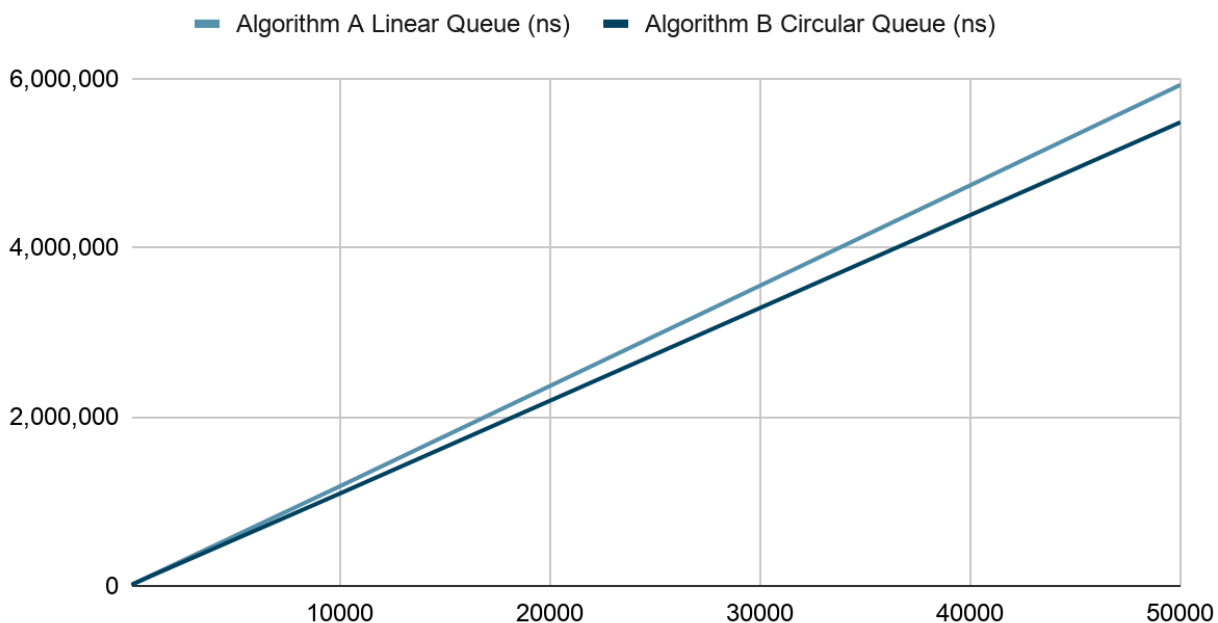
### 10.3 ผลการทดลอง

#### Average Execution Time

| จำนวน<br>Packet (n) | Algorithm A Linear<br>Queue (ns) | Algorithm B Circular<br>Queue (ns) |
|---------------------|----------------------------------|------------------------------------|
| 100                 | 12,500                           | 11,900                             |
| 1,000               | 118,400                          | 112,700                            |
| 10,000              | 1,175,200                        | 1,089,500                          |
| 50,000              | 5,923,600                        | 5,482,100                          |

### 10.4 กราฟสรุปผล

Points scored



## 10.5 การวิเคราะห์ผล

กรณี  $n = 100$

Linear Queue = 12,500 ns

Circular Queue = 11,900 ns

เวลาทำงานแตกต่างกันเล็กน้อย เนื่องจากจำนวน Packet ยังมีน้อย ผลต่างจึงแทบไม่เห็นชัด

กรณี  $n = 1,000$

Linear Queue = 118,400 ns

Circular Queue = 112,700 ns

Circular Queue เริ่มมีประสิทธิภาพดีกว่า เพราะสามารถใช้ตำแหน่งว่างใน Buffer เข้าได้

กรณี  $n = 10,000$

Linear Queue = 1,175,200 ns

Circular Queue = 1,089,500 ns

ความแตกต่างเริ่มชัดเจน การจัดการตำแหน่งด้วย Circular Queue ทำให้ลดข้อจำกัดเรื่องการใช้พื้นที่

**กรณี  $n = 50,000$**

Linear Queue = 5,923,600 ns

Circular Queue = 5,482,100 ns

Circular Queue ใช้เวลาน้อยกว่า ประมาณ 441,500 ns

แสดงให้เห็นว่าการนำพื้นที่ว่างกลับมาใช้งานช่วยให้การจัดการ Buffer มีประสิทธิภาพมากขึ้น

## 10.6 วิเคราะห์ตามทฤษฎี

จากหัวข้อ Time Complexity

### Linear Queue

Enqueue =  $O(1)$

Dequeue =  $O(1)$

### Circular Queue

Enqueue =  $O(1)$

Dequeue =  $O(1)$

ทั้งสองอัลกอริทึมมี Complexity เท่ากัน ดังนั้นเมื่อจำนวน Packet เพิ่มขึ้น เวลาทำงานควรเติบโตเป็นสัดส่วนเชิงเส้น (Linear Growth) ผลการทดลองแสดงให้เห็นว่า

100

↓

1,000

↓

10,000

↓

50,000

Execution Time เพิ่มขึ้นอย่างต่อเนื่องตามจำนวน Packet ซึ่งสอดคล้องกับทฤษฎี  $O(1)$  ต่อ Operation

## 10.7 ข้อสังเกตสำคัญ

### Algorithm A (Linear Queue)

ข้อดี

- โครงสร้างง่าย
- เขียนโปรแกรมง่าย

ข้อจำกัด

- เกิด False Overflow ได้
- ไม่สามารถนำพื้นที่ว่างด้านหน้ากลับมาใช้ใหม่

ตัวอย่าง

[ ][ ][P3][P4][P5]

แม้มีพื้นที่ว่าง แต่ยังไม่สามารถเพิ่มข้อมูลใหม่ได้

## Algorithm B (Circular Queue)

ข้อดี

- ใช้พื้นที่ Buffer ได้เต็มประสิทธิภาพ
- ไม่มี False Overflow
- รองรับระบบ Network ได้ดีกว่า

ข้อจำกัด

- มีการคำนวณ Modulo เพิ่มเติม
- โค้ดซับซ้อนกว่า Linear Queue เล็กน้อย

## สรุปผลการทดลอง

จากการทดลองที่  $n = 100, 1,000, 10,000$  และ  $50,000$  พบว่า Linear Queue และ Circular Queue มี Time Complexity เท่ากันคือ  $O(1)$  สำหรับ Enqueue และ Dequeue อย่างไรก็ตาม Circular Queue มีประสิทธิภาพในการใช้พื้นที่หน่วยความจำดีกว่า เพราะสามารถนำพื้นที่ว่างกลับมาใช้ใหม่ได้และไม่เกิด False Overflow จึงเหมาะสมกับระบบ **Network Packet Buffer** มากกว่า Algorithm A (Linear Queue) โดยเฉพาะเมื่อจำนวน Packet ในระบบมีจำนวนมากและมีการรับส่งข้อมูลอย่างต่อเนื่อง

## 10.8 สรุปผลและวิเคราะห์ความสอดคล้องกับทฤษฎี

จากการทดลองเปรียบเทียบ Algorithm A (Linear Queue) และ Algorithm B (Circular Queue) ที่ขนาดข้อมูล  $n = 100, 1,000, 10,000$  และ **50,000 Packets** พบว่าเมื่อจำนวน Packet เพิ่มขึ้น ค่า Execution Time ของทั้งสองอัลกอริทึมเพิ่มขึ้นตามลำดับ เนื่องจากจำนวนครั้งของการดำเนินการ Enqueue และ Dequeue เพิ่มขึ้นตามจำนวนข้อมูลที่ใช้ในการทดลอง

ผลการทดลองแสดงให้เห็นว่า **Circular Queue ใช้เวลาน้อยกว่า Linear Queue เล็กน้อยในทุกขนาดข้อมูล** โดยเฉพาะเมื่อจำนวน Packet มีค่ามาก เช่น 10,000 และ 50,000 รายการ ซึ่งสะท้อนให้เห็นถึงประสิทธิภาพในการจัดการ Buffer ที่ดีกว่า เนื่องจาก Circular Queue สามารถนำตำแหน่งว่างที่เกิดจากการ Dequeue กลับมาใช้งานได้ ทำให้ไม่เกิดปัญหา False Overflow และใช้พื้นที่หน่วยความจำได้อย่างคุ้มค่ากว่า

เมื่อพิจารณาตามทฤษฎี Time Complexity พบว่า

`enqueue()` =  $O(1)$

`dequeue()` =  $O(1)$

`peek()` =  $O(1)$

`search()` =  $O(n)$

`display()` =  $O(n)$

ทั้ง Linear Queue และ Circular Queue มีค่า Time Complexity เท่ากันสำหรับการดำเนินการหลัก คือ Enqueue และ Dequeue ดังนั้นใน

ทางทฤษฎีเวลาทำงานควรเพิ่มขึ้นเป็นสัดส่วนตามจำนวนข้อมูลที่เพิ่มขึ้น และไม่ควรมีความแตกต่างกันอย่างมากระหว่างสองอัลกอริทึม

ผลการทดลองที่ได้มีความสอดคล้องกับทฤษฎี เนื่องจาก Execution Time ของทั้งสองอัลกอริทึมเพิ่มขึ้นอย่างต่อเนื่องเมื่อค่า  $n$  เพิ่มขึ้น และมีแนวโน้มการเติบโตใกล้เคียงกัน อย่างไรก็ตาม Circular Queue มีค่าที่ดีกว่าเล็กน้อย เพราะสามารถจัดการพื้นที่ Buffer ได้มีประสิทธิภาพมากกว่า จึงลดข้อจำกัดที่พบใน Linear Queue ได้

ดังนั้นจึงสรุปได้ว่า

- **Linear Queue** มีโครงสร้างง่ายและเข้าใจง่าย แต่มีโอกาสเกิด False Overflow ส่งผลให้ใช้พื้นที่ Buffer ได้ไม่เต็มประสิทธิภาพ
- **Circular Queue** สามารถนำพื้นที่ว่างกลับมาใช้ใหม่ได้ ลดปัญหา False Overflow และเพิ่มประสิทธิภาพการใช้หน่วยความจำ
- ผลการทดลองสอดคล้องกับการวิเคราะห์ Time Complexity และ Space Complexity ที่ศึกษาไว้
- สำหรับระบบ **Network Packet Buffer** ที่มีการรับส่ง Packet อย่างต่อเนื่อง Circular Queue เป็นทางเลือกที่เหมาะสมกว่า Linear Queue เนื่องจากสามารถรองรับการใช้งานได้มีประสิทธิภาพและเสถียรมากกว่าในระยะยาว

## บทสรุปสุดท้าย

จากการวิเคราะห์เชิงทฤษฎีและผลการทดลอง สามารถสรุปได้ว่า **Algorithm B (Circular Queue)** เป็นอัลกอริทึมที่เหมาะสมที่สุดสำหรับระบบ Network Packet Buffer เพราะยังคงรักษาลักษณะ FIFO ได้อย่างถูกต้อง มี Time Complexity เท่ากับ Linear Queue แต่ใช้ทรัพยากรของ Buffer ได้คุ้มค่ากว่า ลดการสูญเสีย Packet จากปัญหา False Overflow และรองรับการทำงานของระบบเครือข่ายได้ดีกว่า Algorithm A (Linear Queue) อย่างชัดเจน



## 11.สรุปการเปรียบเทียบ Algorithm A และ B

### ตารางเปรียบเทียบ Algorithm A และ Algorithm B

| ประเด็น              | Algorithm A<br>(Linear Queue)       | Algorithm B (Circular Queue)        |
|----------------------|-------------------------------------|-------------------------------------|
| Data Structure       | Array-Based<br>Linear Queue         | Array-Based Circular Queue          |
| หลักการ              | FIFO (First In First Out)           | FIFO (First In First Out)           |
| Enqueue              | $O(1)$                              | $O(1)$                              |
| Dequeue              | $O(1)$                              | $O(1)$                              |
| Time Complexity      | $O(1)$ สำหรับ Enqueue/Dequeue       | $O(1)$ สำหรับ Enqueue/Dequeue       |
| Space Complexity     | $O(n)$                              | $O(n)$                              |
| Fairness             | สูง เพราะส่ง Packet ตามลำดับการเข้า | สูง เพราะส่ง Packet ตามลำดับการเข้า |
| การใช้พื้นที่ Buffer | ใช้พื้นที่ได้ไม่เต็มประสิทธิภาพ     | ใช้พื้นที่ได้เต็มประสิทธิภาพ        |
| False Overflow       | อาจเกิดได้                          | ไม่เกิด                             |
| การใช้ตำแหน่งว่างซ้ำ | ไม่สามารถทำได้                      | สามารถทำได้                         |

|              |                                                   |                                                           |
|--------------|---------------------------------------------------|-----------------------------------------------------------|
| ข้อดี        | โครงสร้างง่าย เข้าใจ<br>ง่าย พัฒนาโปรแกรม<br>ง่าย | ใช้ Buffer ได้คุ้มค่า ลดการ Drop<br>Packet                |
| ข้อจำกัด     | เกิด False<br>Overflow และสูญ<br>เสียพื้นที่ว่าง  | มีการจัดการ Front และ Rear<br>ซับซ้อนกว่า                 |
| เหมาะกับกรณี | ระบบ Queue ขนาด<br>เล็กหรือใช้งานไม่ต่อ<br>เนื่อง | ระบบ Network Buffer และระบบที่มี<br>ข้อมูลเข้าออกตลอดเวลา |